

The Life of the Node — Everything Working Together (Extended)

One worked example — Alice pays Bob 30 HTR — followed in full through every subsystem, from the connection that carries it to the block that buries it as history.

SUBJECT	hathor-core · Capstone
CHAPTER	44 · Capstone
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

End-to-end · Life of a connection · Life of a transaction · Life of a block · Nano-contract call
· Synthesis · Alice pays Bob

Table of Contents

Chapter 44 — The Life of the Node: Everything Working Together	4
Part 1 — The Life of a Connection	6
1.1 Boot and assembly	6
1.2 Finding a peer	6
1.3 The handshake	7
1.4 Catching up, then relaying	7
Part 2 — The Life of a Transaction	9
2.1 The wallet builds the transaction	9
2.2 Signing — proving ownership	9
2.3 Proof-of-work — the anti-spam toll	10
2.4 Serialization — becoming bytes	10
2.5 Propagation — onto the network	11
2.6 Ingestion — the vertex handler	11
2.7 Verification — is it valid on its own?	11
2.8 Consensus — which history does it belong to?	12
2.9 Storage — writing it down	12
2.10 Indexes — making it findable	13
2.11 Announcement — telling the interested	13
2.12 The loop closes — the wallet sees its change	13
Part 3 — The Life of a Block	15
3.1 The template — a node-built candidate	15
3.2 Mining and submission — work done elsewhere	15
3.3 Consensus by score — and the possibility of a reorg	15
3.4 Confirming Alice's payment	16
3.5 Two machineries only blocks drive	16
Part 4 — The Life of a Nano-Contract Call	17

4.1 Accepted on arrival, but not run	17
4.2 Executed at block consensus	17
4.3 State, verifiability, and the cost of failure	18
Recap	18

Chapter 44 — The Life of the Node: Everything Working Together

What this chapter does

- Takes one concrete event — **Alice sends Bob 30 HTR** — and follows it through every subsystem the book has described, so the machine is finally seen running as a whole rather than as parts.
- Traces four lifecycles in turn: a **connection** (the stage everything happens on), a **transaction** (the spine), a **block** (told as the differences from the transaction), and a **nano-contract call** (the one path that breaks the usual rules).
- Re-derives nothing from first principles. Each step gives enough framing to follow it standing alone, describes what happens to our transaction and why, then points to the chapter that proves it (→ Ch N). Read this for the thread; read the cited chapter for the depth.

Every chapter until now took one subsystem and opened it up on a workbench. This one does the reverse: it keeps a single event moving and lets you watch each subsystem do its part as the event passes through. The value here is the thread — the way the pieces hand off to one another — not the internals, which you already have. A developer who reads only this chapter should finish able to narrate how a payment lives and dies inside the node; a developer who wants to know why any single step behaves as it does has forty-three chapters waiting behind the pointers.

We anchor everything on one scenario, fixed now and carried to the end. It is deliberately small — two inputs, two outputs — because a small trace you can hold in your head teaches more than a large one you cannot.

The scenario. Alice's wallet holds two unspent outputs locked to her address: one worth **20 HTR** and one worth **15 HTR** (35 total). She wants to pay **Bob 30 HTR**. Her node is `N_A`; Bob runs `N_B`; around them is the rest of the network. We follow the payment from the moment her wallet decides to spend, to the moment a block buries it as settled history — and then, in Part 4, we follow a different kind of transaction that does not finish when it is accepted.

Alice's transaction "pay Bob 30"

INPUTS (35 consumed)

← her 20-HTR output
← her 15-HTR output

in: $20 + 15 = 35$

OUTPUTS (35 created)

30 HTR → locked to Bob	(change)
5 HTR → locked to Alice	

out: $30 + 5 = 35$ (conserved)

A note on the order of the four parts. We start with the connection because nothing else can happen until a node is woven into the network; then the transaction, which is the richest and most representative trace; then the block, which we tell only as its differences from the transaction so as not to repeat ourselves; and finally the nano-contract call, which earns its own part precisely because it violates an assumption the first three parts quietly rely on.

Part 1 — The Life of a Connection

A transaction needs somewhere to travel. Before Alice's payment can reach Bob or anyone else, her node must already be a living member of the network: booted, connected to peers, and caught up with the shared ledger. This part follows `N_A` from a cold start to a standing, synced link — the stage on which the rest of the chapter is performed.

1.1 Boot and assembly

`N_A` comes to life when an operator runs `hathor-cli run_node`. That single command is routed through a dictionary of subcommands to the `run-node` path, its flags are parsed, and — the first consequential decision — the **settings profile** for the chosen network is loaded and frozen (→ Ch 21, 22). This profile fixes the genesis vertices, the timing constants, the consensus rules: everything that makes "mainnet" a different network from "testnet." Get it wrong and `N_A` would not be a broken node, it would be a node speaking a different language, unable to agree with anyone. The Twisted **reactor** — the one event loop the whole node will run on — is created at this point but left idle, like starting a car's electrical system before putting it in gear (→ Ch 16, 23).

The node then assembles itself. A builder constructs each subsystem in dependency order and wires them into a single coordinating object, the `HathorManager`: the RocksDB storage opens, the indexes are prepared, the wallet loads, the pub-sub bus is created, and the verification and consensus services are built (→ Ch 24). Construction is separate from operation on purpose — the builder makes the parts, the manager runs them.

Starting is its own careful sequence. The manager first checks a flag in storage: if the last shutdown was unclean, the node refuses to start rather than risk acting on metadata that may be corrupt. It then loads the genesis, rebuilds its in-memory view of the ledger from disk, brings each subsystem online in order, and marks itself `READY` (→ Ch 29). The final act of boot is to hand the one thread to the reactor and let it run. From here `N_A` is reactive: it sleeps until something happens — a peer connects, a timer fires, bytes arrive — and responds.

1.2 Finding a peer

A node just born knows no one. The connections manager solves this two ways at once: it reads a small bootstrap list of well-known peers, and it asks the peers it does reach for the addresses they know, so the node's view of the network grows by word of mouth (→ Ch 34). For each candidate it opens an outbound TCP connection, and each

connection becomes its own protocol object living on the reactor — one of potentially thousands, all multiplexed onto the single thread because the node never blocks waiting on any one of them (→ Ch 2, 16).

1.3 The handshake

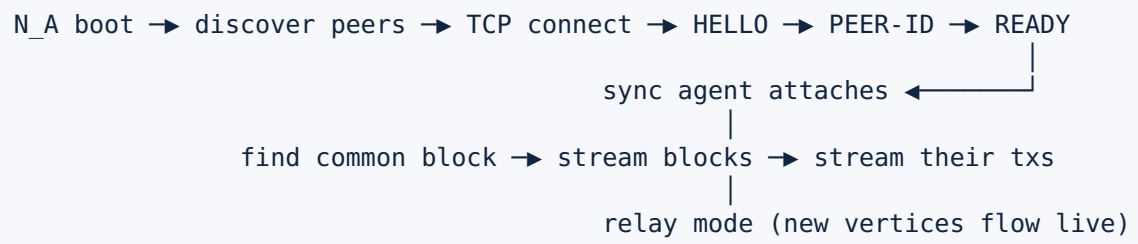
A raw TCP connection is not yet a peer. Each connection climbs a three-step state machine before it is trusted to carry ledger data (→ Ch 34). In **HELLO**, the two nodes exchange protocol and sync versions, confirm they are on the same network, and check that they share the same genesis — a mismatch here means the two are not even trying to maintain the same ledger, and the connection is dropped. In **PEER-ID**, each side proves its identity, which lets the node recognise peers it has seen before and avoid connecting to the same one twice. Only when both sides have completed both steps does the connection transition to **READY**.

It is worth pausing on why a payment system bothers with this much ceremony before exchanging a single transaction. The handshake is the network's immune system: the network check keeps incompatible chains apart, the identity check resists a peer pretending to be many, and the version negotiation guarantees both sides speak one agreed protocol. A connection that cannot satisfy all of it is not an error to be logged and retried in panic — it is only a door that did not open, and the node moves on to the next.

1.4 Catching up, then relaying

The instant a connection reaches **READY**, a **sync agent** attaches to it and the two nodes reconcile their ledgers (→ Ch 35). If **N_A** is behind — a fresh node may have almost nothing while the network has years of history — the agent first finds the most recent block the two peers share, using an n-ary search over block heights that converges in a handful of round-trips rather than walking the whole chain. It then streams the missing blocks in height order, the block backbone first, and for each block streams the transactions that block confirms. Every vertex received this way is fed into the ingestion pipeline that Part 2 is about to follow in detail; sync is not a special path, it is a firehose feeding the ordinary one.

Once **N_A** has caught up, the agent shifts the connection into **relay** mode. From now on, any new vertex either node learns about is pushed across the link in real time. This is the hinge between Part 1 and Part 2: the standing, synced, relaying connections built here are the very wires along which Alice's payment will travel the moment her wallet lets it go.



Part 2 — The Life of a Transaction

Now the actor walks onto the stage. We follow Alice's payment from the instant her wallet decides to spend, through the proofs that make it acceptable, across the network, and through the pipeline every node runs to absorb it — ending where it began, at a wallet whose balance has changed. This is the spine of the entire node and the longest trace in the chapter, so we take it one subsystem at a time.

2.1 The wallet builds the transaction

A wallet's quiet, constant job is to know which coins it can spend. Alice's wallet watches the ledger and keeps her unspent outputs in view, so when she asks to send 30 HTR it already has the raw materials at hand (→ Ch 40, 28). Its first task is **input selection**: choose a set of her unspent outputs whose values cover the payment. It picks the 20-HTR and the 15-HTR outputs — 35 total, enough to cover 30. Because outputs in a UTXO ledger are spent whole and cannot be split, the 5 HTR of surplus does not vanish; the wallet creates a second output of 5 HTR locked back to Alice's own address, the **change** (→ Ch 7).

The transaction now has a definite shape: two **inputs**, each a pointer of the form (prior transaction id, output index) identifying exactly one of Alice's coins, and two **outputs**, one of 30 HTR carrying a lock that only Bob's key can open and one of 5 HTR carrying a lock only Alice's key can open. The wallet also selects two **parents** for the transaction — current tips of the DAG — because in Hathor a transaction does not wait to be packed into a block; it attaches itself directly to the graph by confirming earlier vertices (→ Ch 8). Keep the two kinds of edge distinct: the inputs say which coins this spends, the parents say which earlier vertices this builds on. They are wholly separate, and conflating them is the classic way to misread the codebase.

At this moment the transaction is structurally finished but worthless to anyone else: it makes a claim on Alice's coins without yet proving she has the right to make it, and it carries no work to deter spam. The next three steps supply the proofs.

2.2 Signing — proving ownership

Each of Alice's inputs spends an output that was locked, when it was created, with a script demanding a valid signature from the key behind her address (→ Ch 7, 31). Ownership in this model is not a name in a ledger row; it is the ability to satisfy that lock. To do so the wallet computes the transaction's **sighash** — a hash taken over the transaction's contents, so that the signature commits to this exact transaction and

cannot be lifted onto another — and signs it with Alice's private key. For each input it produces an unlocking payload of `<signature> <public-key>`, which it attaches to that input (→ Ch 40).

The asymmetry here is the whole foundation of the system. Anyone can verify Alice's signature using her public key, which is published openly; only Alice, holding the matching private key, could have produced it. So the transaction now carries, for each coin it spends, cryptographic proof that its rightful owner authorised the spend — without that owner ever revealing the secret that proves it.

2.3 Proof-of-work — the anti-spam toll

In most blockchains only blocks carry proof-of-work; in Hathor, every transaction does too, because a transaction attaches itself directly to the ledger and so must pay a small toll for the privilege (→ Ch 9). The wallet computes the minimum **weight** the transaction must meet — a value derived from its size and amount — and then grinds a **nonce**, hashing the transaction over and over with different nonce values until its hash falls below the target that weight implies (→ Ch 37). The cost is deliberately modest for an honest single payment and ruinous for anyone trying to flood the DAG with millions of junk transactions.

Because this grinding is a burst of pure computation rather than waiting on the network, doing it on the node's single reactor thread would freeze every other connection while it ran — so it is pushed onto a thread pool and done off to the side (→ Ch 2, 16). When a satisfying nonce is found, the transaction's hash is now fixed, and that hash is its identity for the rest of its life: the thing every node will use to name it, look it up, and refer to it.

2.4 Serialization — becoming bytes

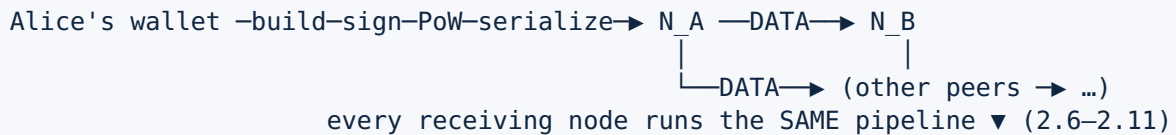
A transaction held as Python objects in Alice's wallet cannot travel a wire or rest on a disk; it must become a flat sequence of **bytes**, and not just any encoding will do (→ Ch 26). The bytes are load-bearing, because the hash that identifies the transaction and the signatures that authorise it are both computed over exactly this byte sequence. Hathor uses a bespoke binary format that lays the fields down in a fixed, deterministic order: the version and signal bits, then the funds section (the token list, the inputs, the outputs), then the graph section (the parents), then the nonce.

Determinism is the reason the format is hand-rolled rather than borrowed from JSON or a general scheme. Every node that serializes this transaction must produce byte-for-byte the same result, because every node must compute the same hash from it and reach the same verdict on it — consensus would be impossible if two honest nodes could encode the same transaction two different ways. The very bytes produced here are what will be handed to the network and, later, written to storage unchanged.

2.5 Propagation — onto the network

Alice's node releases the serialized transaction to its peers over the standing relay connections established in Part 1 (→ Ch 34). On the wire it is a single framed message — a **DATA** message carrying the transaction's bytes — sent to each connected peer. Those peers forward it to their peers, and within moments the transaction has rippled across the network, reaching Bob's node **N_B** after a hop or two.

There is no privileged route and no special recipient. Every node that receives the transaction — Bob's, and every other — runs the identical acceptance pipeline that we follow next; the network has no centre, so "being accepted" means each node independently arrives at the same conclusion. From here we watch the transaction arrive at one node, **N_B**, with the understanding that the same thing is happening everywhere at once.



2.6 Ingestion — the vertex handler

When the bytes arrive at **N_B**, they are parsed back into a transaction object and handed to the **vertex handler**: the single, narrow chokepoint where "a vertex arrived" turns into "the ledger changed" (→ Ch 33). Funnelling every new vertex — whether freshly received or downloaded during sync — through one place is what keeps the node's behaviour consistent and its bugs findable.

The handler's first job is triage. Has it seen this transaction before? If so, it stops; there is nothing to do. Are the things the transaction depends on — the outputs its inputs spend, the parents it names — all present on this node? If a dependency is missing, the transaction cannot be judged yet, and it waits until sync delivers what it lacks, so that a vertex is never evaluated before the things it stands on. For Alice's transaction, assume **N_B** already holds her two spent outputs and the parents she chose; the handler proceeds, orchestrating the next three steps as one ordered unit — verify, then run consensus, then save and announce.

2.7 Verification — is it valid on its own?

Verification asks one bounded question: taken entirely by itself, ignoring everything else on the ledger, does this transaction obey the protocol's rules (→ Ch 31)? The node re-computes the transaction's hash and confirms its proof-of-work clears the target. It checks the structural rules and the minimum weight. For each input, it runs the input's unlocking script together with the spent output's locking script on a small **stack machine** — and this is the precise moment Alice's signatures are tested against the

public keys they claim to match; a forged signature, a tampered output, a spend of someone else's coin, all fail right here (→ Ch 31). It confirms **conservation**: the inputs total 35, the outputs total 35, and no value has been conjured from nothing.

A transaction that fails any of these checks is rejected outright and never enters the ledger — there is no appeal, because the failure is in the transaction itself. Note the careful scope of this stage: it can prove a transaction internally valid without yet knowing whether it belongs in the canonical history, because that second question depends on everything else and is consensus's job, not verification's. Alice's transaction is well-formed, correctly signed, and balanced, so it clears verification and moves on.

2.8 Consensus — which history does it belong to?

Verification proved the transaction is valid; consensus decides whether it is canonical — part of the one history the network agrees on (→ Ch 32). For a plain payment the decisive question is conflict: does any other transaction already in the DAG spend either of the two specific outputs Alice is spending? In the normal case the answer is no. Alice's transaction is accepted as **executed**, its metadata is written to record that it is not voided, and the two outputs it consumes are marked as spent by it.

The interesting case is the one Alice is not in: a **double-spend**, where two transactions each try to consume the same output. The DAG holds both at once — its structure does not forbid the conflict — and consensus resolves it by comparing the **accumulated weight** behind each, letting the heavier history win and marking the lighter transaction voided; on an exact tie, both stay voided until some later vertex tips the balance (→ Ch 32). Voiding is a mark, not a deletion, which is what allows the decision to be reversed later if the weights shift. Because Alice spent her coins exactly once, in good faith, her transaction faces no rival and takes its place in the canonical ledger uncontested.

2.9 Storage — writing it down

With the transaction verified and its consensus state decided, the node makes it durable (→ Ch 27). Hathor's storage is an embedded key-value store split into separate **column families**, and the transaction is written across three of them: its serialized body (the very bytes from §2.4) under its hash in one family, its mutable metadata — the consensus verdict, the accumulated weight, the spent markers — in a second, and its immutable computed metadata in a third. Keeping the unchanging body apart from the evolving metadata lets each be read and rewritten without disturbing the other.

One detail repays attention. The two outputs Alice spent are not erased; they are recorded as spent by this transaction. Marking rather than deleting is what makes a future reorganization survivable: if consensus ever changes its mind about this region

of history, the spend can be cleanly undone and the coins returned to the unspent set, because the record of what they were never went away. Once written, the transaction is permanent — a restart of the node will find it exactly here, byte for byte.

2.10 Indexes — making it findable

Storing the transaction under its hash answers the question "give me the vertex with this hash," but that is not a question wallets ask. Bob's wallet wants to know "what can I spend now?" — a question the raw store cannot answer without scanning everything. So the node maintains **derived indexes** that precompute the answers, and it updates them to reflect the new reality (→ Ch 28). The **UTXO index**, keyed by address, removes Alice's two now-spent outputs and adds the two new ones: a 30-HTR output recorded against Bob's address and a 5-HTR change output recorded against Alice's. The **address index** notes that both addresses have new history.

Nothing in the indexes is independent truth — every entry could be rebuilt from scratch by replaying the stored transactions, which is exactly what the node does when it initialises after a restart. Their entire purpose is speed: they convert a question that would otherwise require walking the whole ledger into a direct lookup, at the cost of the small bookkeeping we just watched.

2.11 Announcement — telling the interested

The transaction is now verified, settled, stored, and indexed; the last step is to tell the parts of the system that care. The node publishes the event on its internal **pub-sub bus**, and whoever subscribed to "a new transaction was accepted" reacts (→ Ch 30). The metrics counters advance (→ Ch 42); any wallet or dashboard watching over a **WebSocket** is pushed a live notification (→ Ch 36); and, if the durable event queue is enabled, the event is appended to a replayable log so that downstream systems can later reconstruct exactly what the node saw and when (→ Ch 30). The publisher knows none of these subscribers individually — it announces into the room, and the interested parties act, which is what lets new subsystems hook into the flow without the core being rewired.

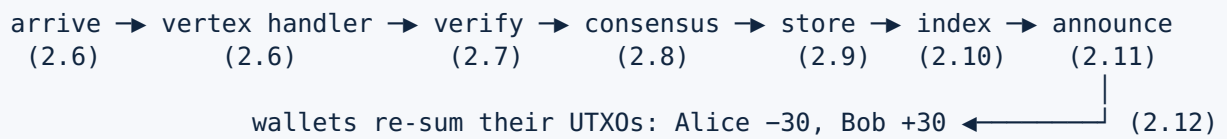
This same acceptance also re-enters the relay path from Part 1: **N_B**, having accepted the transaction, now forwards it to its peers, which accept and forward in turn. That outward ripple is how a payment that began in one wallet comes, within seconds, to be held by the entire network.

2.12 The loop closes — the wallet sees its change

The journey ends where it started, back at a wallet. Alice's wallet subscribes to the same notifications, so when her own node accepts the transaction it sees the two facts that concern her: her 20-HTR and 15-HTR outputs are now spent and gone, and a fresh

5-HTR change output is now hers (→ Ch 40). Bob's wallet, on his node, sees a new 30-HTR output it controls appear. Crucially, neither wallet stores a number called "balance" — each re-sums the unspent outputs it can spend, and after this transaction those sums have moved by exactly 30 HTR. The payment has, to all appearances, happened.

And yet it is not final. A transaction accepted into the DAG is canonical for now, but its place rests only on the work accumulated around it, which is still thin. To turn "accepted" into "settled beyond practical doubt" takes a block — and that is the next trace.



Part 3 — The Life of a Block

A block walks much of the same road as a transaction: it too is a vertex, verified, run through consensus, and stored. Re-telling that shared road would only repeat Part 2, so this part follows the **differences** — the handful of things a block does that a transaction does not. The block we follow is the one whose arrival will finally confirm Alice's payment and begin turning it from "accepted" into "irreversible."

3.1 The template — a node-built candidate

The first difference is in who builds it. Alice's transaction was assembled by her wallet; a block is assembled by the node, on demand, as a **template** for miners to solve (→ Ch 37). To build one, the node selects the block's parents — including, always, the previous block, which keeps the block backbone an unbroken chain — adds the **reward** outputs that mint brand-new HTR into existence, and computes the minimum **weight** the block must meet. That weight is set by the difficulty-adjustment algorithm, which nudges it up or down to keep blocks arriving at the network's target rhythm of roughly one every 30 seconds regardless of how much mining power is at work (→ Ch 9). The finished template is a fill-in-the-blank puzzle: everything is fixed except the nonce.

3.2 Mining and submission — work done elsewhere

The second difference is scale and location. Alice's transaction carried a small proof-of-work her own wallet could grind in a moment; a block's proof-of-work is enormous and is done by dedicated **miners** outside the node — specialised hardware speaking the Stratum protocol, or a merged-mining arrangement that earns Hathor blocks as a by-product of mining Bitcoin (→ Ch 37). A miner takes the template and hashes it with nonce after nonce, billions of times, until one produces a hash beneath the block's hard target. The moment it succeeds, it submits the solved block back to the node — which then feeds it into the same ingestion pipeline Alice's transaction travelled: the vertex handler receives it, verification checks it, consensus weighs it, storage records it. The road is shared; only the rules layered on top differ.

3.3 Consensus by score — and the possibility of a reorg

Here lies the deepest difference. A transaction's consensus question was local — "does this conflict with another spend?" A block's question is global — "does this change which chain is the true one?" (→ Ch 32). Blocks compete by **score**, the total accumulated work of the sub-DAG behind them, and the chain whose tip has the highest score is the canonical history by definition. When our block extends the current best chain, it becomes the new tip and little drama follows.

But if, while our block was being mined, a competing branch had quietly accumulated more work, then accepting our block forces a **reorganization**: the node abandons the lighter branch for the heavier one, marks the blocks and transactions unique to the abandoned branch as voided, and re-applies any transactions that still belong on the new branch (→ Ch 32). This is the concrete machinery behind a phrase from early in the book — that finality is only ever probabilistic. A transaction one or two blocks deep could, in principle, still be undone by a large enough reorg; what protects it is not a guarantee but accumulating cost.

3.4 Confirming Alice's payment

The block we are following conflicts with nothing and extends the best chain, and in doing so it reaches back through its parents into the region of the DAG that holds Alice's transaction, **confirming** it (→ Ch 8, 9). This is the event that changes the character of her payment. Before the block, Alice's transaction was accepted but lightly anchored; now a block's full weight sits on top of it, and every further block mined onto this chain piles more accumulated work above it. To reverse Alice's payment, an attacker would have to privately out-mine the entire honest network from before her transaction forward — a cost that climbs so steeply with each new block that after a handful of confirmations Bob can treat the 30 HTR as unconditionally his. "Probabilistic finality" is, in practice, certainty that deepens with every block.

3.5 Two machineries only blocks drive

A block does two further things no transaction ever does, and both are easy to miss because they happen quietly at acceptance. The first is **feature activation**: each block carries signal bits, and accepting the block folds those bits into a rolling tally that decides whether a proposed protocol upgrade has gathered enough sustained miner support to lock in and, eventually, switch on — the mechanism by which the network changes its own rules without a disruptive flag-day (→ Ch 38). The second is **nano-contract execution**: any contract calls confirmed by this block are run now, as part of the block's consensus, in a deterministic order, with their results committed to contract state (→ Ch 39, and Part 4). A block, then, is more than a stack of confirmed payments — it is the heartbeat that advances the network's slow, deliberate machinery one step with every tick.

```
Transaction trace (Part 2):  build – verify – consensus(conflict?) – store – announce
The block trace adds/changes:
  • template built by the NODE, with reward minting           (3.1)
  • EXTERNAL mining (Stratum / merged), then submission      (3.2)
  • consensus by SCORE – may trigger a REORG                 (3.3)
  • CONFIRMS the transactions beneath it (Alice's payment)   (3.4)
  • advances feature signalling + runs nano-contracts         (3.5)
```

Part 4 — The Life of a Nano-Contract Call

The fourth trace earns its own part because it breaks a rule the first three quietly assumed: that a vertex's effect is settled the instant consensus accepts it. A nano-contract call is, on the surface, just another transaction — it is built, signed, serialized, propagated, verified, consensus-checked, and stored exactly as Alice's payment was, travelling the whole of Part 2. But what the contract actually does happens later, and somewhere else, and that gap is the point of this part.

4.1 Accepted on arrival, but not run

A nano-contract transaction carries, in a dedicated header, the identity of the contract it calls and the method to invoke, along with any tokens it deposits into or withdraws from the contract. When such a transaction is accepted, the node records the intention to make that call — but it does **not** execute the method then and there (→ Ch 39). The reason is the same caution that runs through the whole consensus design: running a state-changing program the moment its transaction arrives, before the network has agreed on that transaction's place in history, would let a call that is about to lose a conflict — or be voided in a reorg — leave its fingerprints on shared contract state. Acceptance, here, means "this call is queued and ordered," not "this call has happened."

4.2 Executed at block consensus

Contracts run when a **block** confirms them, as part of that block's consensus (→ Ch 39, 32). A block imposes a definite order, and the calls it confirms are executed in a deterministic, seeded sequence, so that every node replaying the same block performs the same calls in the same order and arrives at exactly the same result — without which, two honest nodes could compute different contract state and the network would fork. Each call is handed to the **runner**, which executes the blueprint's method against the contract's current state. A resource-metering layer is meant to bound how much computation a single call may consume — a defence against a contract that loops forever or tries to exhaust the node — though on the current branch that bound is scaffolded but not yet enforced, even as the sandbox that restricts what a contract is allowed to touch is fully active (→ Ch 39). This is the one place in the entire node where the "execute on arrival" model that governed Alice's payment is deliberately set aside in favour of "execute on confirmation."

4.3 State, verifiability, and the cost of failure

A contract's state does not live in outputs the way Alice's coins do; it lives in a verifiable **Merkle/Patricia trie**, one per contract, whose root is anchored into the confirming block (→ Ch 39). A call that succeeds commits its changes into that trie and advances the contract. A call that fails — exhausts its intended budget, breaks a rule, raises an error mid-execution — is marked with a dedicated voiding identifier, its partial state changes are rolled back as if they never happened, and the failure itself is recorded rather than allowed to corrupt the contract (→ Ch 32, 39). Because the state is trie-backed and its root sits inside the block, two nodes can prove they hold identical contract state by comparing roots alone — the very same verifiability the rest of the ledger earns from its hashes, now extended to running programs.

Recap

Lifecycle	Spans	The one thing to remember
Connection (Part 1)	Ch 21–24, 29, 34, 35	a synced, relaying link is the stage every vertex travels on
Transaction (Part 2)	Ch 7–9, 26, 28, 30–34, 37, 40	arrive → verify → consensus → store → index → announce, at every node identically
Block (Part 3)	Ch 9, 32, 37, 38, 39	same pipeline, but consensus is by score, and a block confirms and triggers
Nano-contract (Part 4)	Ch 32, 39	accepted on arrival, but executed later, at block consensus, against trie state

Follow one small payment all the way through and the node stops being a catalogue of packages and resolves into a single machine. A wallet proves ownership with a signature; a binary format makes that proof reproducible on every node; a network of handshaken, synced connections carries it; a pipeline verifies, agrees on, records, indexes, and announces it; a block buries it; and the chain of blocks that follows makes burying it irreversible. Every subsystem in this book exists to play one part in that sequence, and the sequence, taken whole, is the node. With these four lifecycles in hand — and the chapter behind each step waiting whenever a detail is wanted — you can open `hathor-core` at any file and know not only what the code in front of you does, but where it sits in the life of the thing passing through it.